Terminal Buddy: Technical Build Specification (V2.0 - Deep Dive)

Lead Architect: Fred **Core Language:** Rust 1.80+ (Stable) **Frameworks:** GPUI (Driver), Tokio (Daemon), Memvid (Memory), MCP (Intelligence)

1. The "Sidecar" Inter-Process Communication (IPC)

The core of Terminal Buddy is the **coupler** between the GUI and the Daemon. We use a **Binary Typed Protocol** (via `bincode`) over Unix Domain Sockets to ensure zero-latency communication.

The Message Bus Schema

```
#[derive(Serialize, Deserialize)]
pub enum BuddyMessage {
    // GUI -> Daemon
    Input { data: Vec<u8> },           // Keystrokes
    VoiceStream { pcm: Vec<f32> },      // Raw Audio for Whisper
    CommandTrigger { cmd: String },    // High-level AI intent

    // Daemon -> GUI
    TerminalOutput { data: Vec<u8> },  // PTY output to render
    SystemDelta {                      // The "Teachable Moment" data
        cpu_spike: f32,
        disk_diff_kb: i64,
        files_changed: Vec<String>,
        exit_code: i32
    },
    BuddyThought { text: String }      // AI explanation for the sidecar panel
}
```

2. The "Cause & Effect" Engine (State Introspection)

This is the "Hardware" level of your request. To track how a command affects the computer, we don't just "guess"; we monitor the kernel.

A. The Pre-Execution Snapshot

Before the PTY executes a command, the Sidecar polls:

1. **File System:** `inotify` (Linux) or `FSEvents` (macOS) is initialized on the current working directory.

2. **Process Tree:** A snapshot of child PIDs is taken via the `sysinfo` crate.
3. **IO Stats:** Current disk read/write counters are cached.

B. The Post-Execution Diff

Upon the `SIGCHLD` signal (command completion):

1. **Delta Calculation:** We compare the snapshot to the current state.
2. **Journaling:** The result is formatted into a "Context Frame."
3. **Teachable Moment Logic:** If a command results in a massive disk drop or a new hidden file, the Daemon flags this event as "High Interest" for the AI.

3. The "Deep Time" Memory (Memvid Integration)

Most terminals use a text file for history. We use a **Write-Ahead Log (WAL)** stored in Memvid frames.

- **Frame Composition:** Each command creates a "Smart Frame" containing:
  - The raw terminal buffer (compressed).
  - The `SystemDelta` (calculated above).
  - A timestamp-link to the computer's system logs (`journalctl` or `/var/log/syslog`).
- **The "One Year" Index:** We use a local vector index (via `lance-db` or similar) to allow the user to query: *"Find the time I updated the SSL certs last year and tell me what changed in the filesystem."*

4. The "Square" (Spatial Dashboard Logic)

You mentioned a "Square" or multiple windows. This requires a **Tiling Engine** built into the Rust GUI.

- **The Multiplexer:** We implement a custom layout engine using `gpui` 's Flexbox-style containers.
- **Synchronized Logging:** Each "Pane" in the square has a unique `SessionID` . The Sidecar logs all panes simultaneously to the same Memvid file, allowing you to see how a server in Pane 1 affected the build in Pane 2.

5. The Control Plane (Neovim & Voice)

We treat the keyboard and voice as a single **Input Stream**.

1. **The Aider Logic:** Voice is processed via **Local Whisper**.

2. **Intent Injection:** If you say "Fix this error," the AI reads the terminal buffer (the "Grid"), generates a diff, and sends it to the **Embedded Neovim** instance via the RPC channel.
3. **Customizability:** Every keybinding and voice trigger is defined in a `buddy.toml`. The user can remap the "Push-to-Talk" to any hardware interrupt.

6. Provable Build Directives (Phase 1)

1. `src/daemon/pty.rs`: Implement the PTY wrapper using `portable-pty`.
2. `src/daemon/sensors.rs`: Implement the `SystemSnapshot` struct and the `inotify` file-watcher.
3. `src/gui/renderer.rs`: Set up the `gpui` window to draw the terminal grid at 120FPS using texture-atlas caching for text.
4. `src/memory/memvid.rs`: Build the bridge that pushes the `BuddyMessage::SystemDelta` into the `.mv2` file format.